DATA/CS 172: Python for Data Analysis

APIs

Prof. Travis Mandel

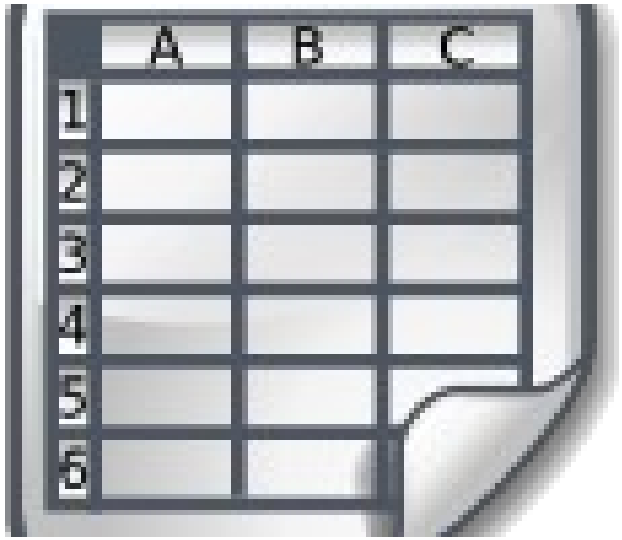
10/21/2024

UNIVERSITY of HAWAI'I® HILO

Announcements

- Lab 13 due today
- Program 2 out Wednesday

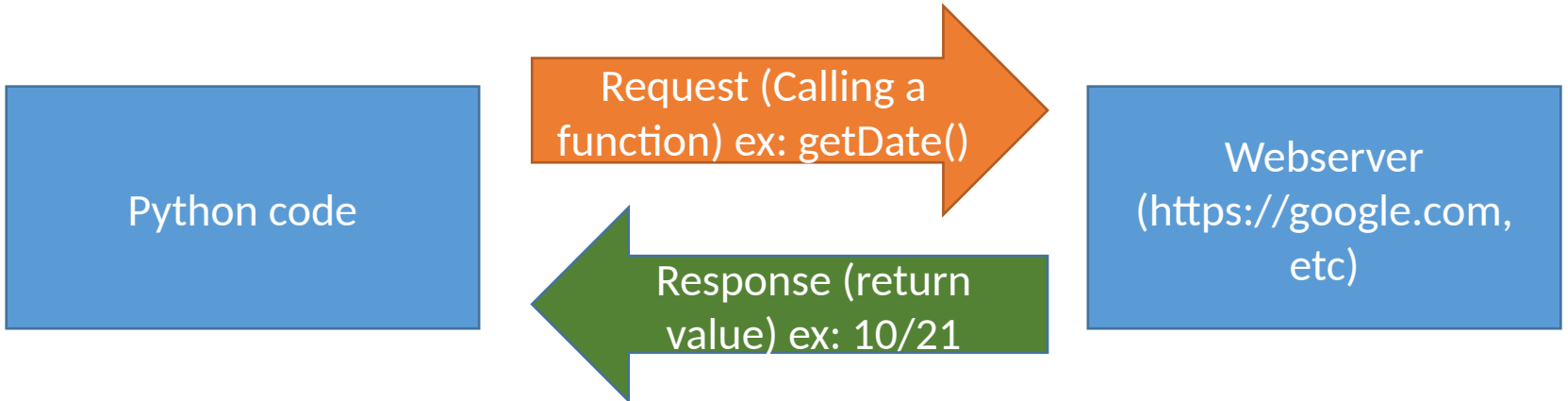
Last time: JSON



- Like Python variables (dicts, lists, etc.)
- Can represent complicated nested structures
- Easy to load and dump given Python variables
  - Some small differences to be aware of

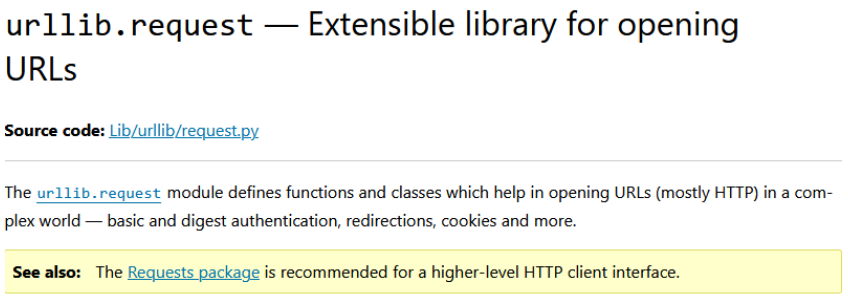
Where does JSON data come from?

- Well, they could come from anywhere…
- But #1 place they come from is calls to Web APIs!
- What is an API?
- API stands for Application Programming Interface
  - Sounds complicated
  - Reality: Basically a set of functions provided by a website
  - “Information hiding” again – but to a greater extent!



Communication over the internet

- To make an API request, need to send info over the internet
- Problem: Python: built in support is limited
  - Urllib.requests not very popular – even the documentation recommends against it
- Solution:
  - Install module via pip



`urllib.request` — Extensible library for opening URLs

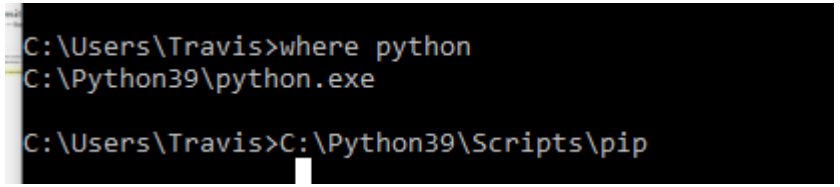
Source code: Lib/urllib/request.py

The `urllib.request` module defines functions and classes which help in opening URLs (mostly HTTP) in a complex world — basic and digest authentication, redirections, cookies and more.

See also: The Requests package is recommended for a higher-level HTTP client interface.

Installing via pip

- Let’s say I want to use a library that I don’t have installed
- Import requests
- Need to use “pip” to install
  - Windows “cmd”/”Command Prompt”
    Mac: Terminal
  - Type “pip install requests”
- Pip not found?
  - Type “where python” (Windows) or “which python” (Mac/Linux)
    - Might show C:\Python312 or something else
  - Need to run PythonDir/Scripts/pip



  - All else fails? See instructions on Laulima (beginning of Modules) about installing pip

Requests module

- import requests
- response = requests.get(url)
- Makes a request to the URL
  - URL is something like https://www.google.com/careers
  - See demo for real-world examples
- Uses HTTP or HTTPS
  - Hypertext transfer Protocol – a way of formatting web requests and responses
  - The S stands for secure – recommended where possible

Common HTTP Status codes

- After the request, can get the status code with response.status_code
- Common codes:
  - 200 – OK
  - 304 Not Modified
  - 400 Bad Request
  - 403 Forbidden
  - 404 – Page not found
  - 500 – Internal Server Error
- **Very important** to always check this in case something has gone wrong!
  - Webserver could go down at any time
  - If it is a bad code, ideal to print response.text in case server has more information on the error

Parsing the response

- Response.text gets the string text
- Can load this with function (learned last time)
  - json.loads()
- Note: Many sources suggest the following instead:
  - response.json()
  - NOT recommended in my opinion/experience – do not use
    - Why? If it is not JSON-formatted, will crash with a nasty error without even letting us see the text!

API Keys

- Most APIs require you to provide a key that you obtain from them first
  - Every request has to send this key
  - Usually requires signing up (sometimes requires credit card number!)
  - Usually have limits or charges
- Why?
  - Server time/resources cost $
  - Getting data from websites is valuable (and becoming more so)
  - Need to prevent cyberattacks
    - Denial of Service (DoS) – keep making huge number of API requests
- Important note on API Keys:
  - API keys should NOT be hardcoded
  - Especially not in code anyone else is going to see (on Github etc.)
  - This could have disastrous consequences
  - Solution: store in a separate file which is not uploaded anywhere

Q: How can we send API Keys?

- This requires sending information from Python to the server
- There are three ways to do this!
  - Covered in next slides
- Many websites may only accept 1-2 of the three ways – need to read API documentation

Method 1 of passing information

- Params in URL
- First one begin with a ?, separated by &
  - Each one generally specified as key=value
- Example:
  - https://api.phonevalidator.com/api/v3/phonesearch?apikey={APIKEY}&phone={PHONE}&type=fake,basic

- What exact keys/properties do I need to specify?
  - Need to check the documentation (these are essentially arguments to the function)
- Benefits:
  - Easy to build url string yourself
    - Although, it is recommended to let requests handle this for you
    - Parameters = {“apikey”: “APIKEY”, “phone”:”PHONE”, “Type”:”fake, basic”}
    - Then add params=parameters to API call
- Drawbacks:
  - Difficult/annoying to send large amounts of information

Method 2 of passing information

- What if we need to send longer information to the server?
- POST
  - Requests.post(url, data=variable)
    - Form-encodes variable (No JSON)
  - OR
  - Requests.post(url, json=variable)
    - JSON-encodes variable
    - Same as data=json.dumps(variable) and then setting a header (see next slide)
  - Which one you use depends on documentation
- Similar to params variable should generally be a dict mapping name to value
  - Check documentation for key names
- REST/RESTful API generally refers to an API that uses HTTP GTE, POST, etc to access resources via URLs in a way that satisfies certain requirements
  - Probably most important is that a RESTful API should generally be stateless
  - Re-doing the same API request should return the same result
    - Example: Requesting “next page” not good practice compared to “requesting page 6”

Method 3 of passing information

- Headers!
- Headers get sent with each request to the server
  - Generally contain initial information, such as what format the response is in, etc.
- Sometimes needs API key or other information passed as a header
- Similar way to specify
- head = {"Content-Type": "application/json"}
- requests.post(…, headers=head)

APIs for AI

- APIs have been around for decades…
  - Idea started in ~2000 and has only gained steam since
  - APIs are a great way to access web datasets, especially ones which update frequently
- Why am I just discussing APIs this semester?
  - AI, especially Large Language Models (LLMs)
  - Many of the best models too large to run on end machine
  - Need to be run “in the cloud”
  - Require APIs to integrate into your code
    - See the lab!

Lab out

- Practice with APIs